

Desarrollo de Software

2024 - 2025

“Ejercicio Guiado 1”

Alejandro Pérez Montes 100499978

Alejandro de Santos Lobo 100499148

Grupo 801 Subgrupo 03



Esta obra se encuentra sujeta a la licencia Creative Commons **Reconocimiento - No Comercial - Sin Obra Derivada**

ÍNDICE GENERAL

1. NOMATIVA DE CÓDIGO	1
1.1. Regla Modificada: <i>argument-rgx</i>	1
1.2. Regla Modificada: <i>class-rgx</i>	1
1.3. Regla Modificada: <i>const-rgx</i>	2
1.4. Regla Modificada: <i>function-rgx</i>	3
1.5. Regla Modificada: <i>method-rgx</i>	3
1.6. Regla Modificada: <i>variable-rgx</i>	4
1.7. Regla Modificada: <i>docstring-min-length</i>	5
1.8. Regla Modificada: <i>max-args</i>	5
1.9. Regla Modificada: <i>max-line-length</i>	6
1.10. Regla Modificada: <i>single-line-if-stmt</i>	6
1.11. Regla Modificada: <i>logging-format-style</i>	7
1.12. Regla Modificada: <i>max-nested-blocks</i>	7
1.13. Regla Modificada: <i>useless-return</i>	8
1.14. Regla Modificada: <i>bare-except</i>	9
1.15. Regla Modificada: <i>max-branches</i>	9
2. EVALUACIÓN DE LA NORMATIVA Y DEL CÓDIGO.	11
2.1. Capturas de pylint con código sin cumplir normativa.	11
2.2. Captura de pylint con código cumpliendo normativa	13
2.3. Evaluación del código	13

1. NOMATIVA DE CÓDIGO

1.1. Regla Modificada: *argument-rgx*

Permite escribir una expresión regular para restringir el dominio de nombres que pueden adoptar los argumentos de las funciones o métodos. En nuestro caso la hemos modificado para que solo se acepten nombres en minúsculas y separados por barras bajas

Regla Original: `argument-rgx=`

Regla Modificada: `argument-rgx=^[a-z][a-z0-9_]*$`

Usos Correctos

```
1 def(posicion_x, posicion_y):  
2     pass  
3  
4 def(nombre):  
5     pass
```

Usos Incorrectos

```
1 def(POSICION_X, POSICION_Y):  
2     pass  
3  
4 def(Nombre):  
5     pass
```

1.2. Regla Modificada: *class-rgx*

Permite escribir una expresión regular para restringir el dominio de nombres que pueden adoptar las clases. En nuestro caso la hemos modificado para que solo se acepten nombres que empiecen en mayúscula y que sigan el estilo PascalCase.

Regla Original: `class-rgx=`

Regla Modificada: `class-rgx=^[A-Z][a-z]+(?:[A-Z][a-z]+)*$`

Usos Correctos

```
1 class TransactionManager:
2     pass
3
4 class Vector:
5     pass
```

Usos Incorrectos

```
1 class transaction_manager:
2     pass
3
4 class vector:
5     pass
```

1.3. Regla Modificada: *const-rgx*

Permite escribir una expresión regular para restringir el dominio de nombres que pueden adoptar las constantes. En nuestro caso la hemos modificado para que solo se acepten nombres en mayúscula y separados por barras bajas

Regla Original: `const-rgx=`

Regla Modificada: `const-rgx=^[A-Z][A-Z0-9_]+$`

Usos Correctos

```
1 CONSTANTE
2 EJEMPLO_CONSTANTE
3 VELOCIDAD_ANGULAR_X
4 PISO_3
```

Usos Incorrectos

```
1 constante
2 ejemplo_constante
3 velocidad_angular_x
4 piso_3
```

1.4. Regla Modificada: *function-rgx*

Permite escribir una expresión regular para restringir el dominio de nombres que pueden adoptar las funciones. En nuestro caso la hemos modificado para que solo se acepten nombres en minúscula y separados por barras bajas

Regla Original: `function-rgx=`

Regla Modificada: `function-rgx=[a-z][a-z0-9_]*$`

Usos Correctos

```
1 def calcular_area():
2     pass
3
4 def funcion():
5     pass
```

Usos Incorrectos

```
1 def CalcularArea():
2     pass
3
4 def FUNCION():
5     pass
6
7 def Funcion():
8     pass
```

1.5. Regla Modificada: *method-rgx*

Permite escribir una expresión regular para restringir el dominio de nombres que pueden adoptar los métodos de las clases. En nuestro caso la hemos modificado para que solo se acepten nombres en minúscula y separados por barras bajas

Regla Original: `method-rgx=`

Regla Modificada: `method-rgx=[a-z][a-z0-9_]*$`

Usos Correctos

```
1 class Cuadrado:
2     def calcular_area()
3         pass
4
5     def metodo()
6         pass
```

Usos Incorrectos

```
1 class Cuadrado:
2     def CalcularArea()
3         pass
4
5     def METODO()
6         pass
7
8     def Metodo()
9         pass
```

1.6. Regla Modificada: *variable-rgx*

Permite escribir una expresión regular para restringir el dominio de nombres que pueden adoptar las variables. En nuestro caso la hemos modificado para que solo se acepten nombres en minúscula y separados por barras bajas

Regla Original: `variable-rgx=`

Regla Modificada: `variable-rgx=^[a-z][a-z0-9_]*$`

Usos Correctos

```
1 ejemplo_variable
2 variable
3 vel_1
```

Usos Incorrectos

```
1 EjemploVariable
2 EJEMPLO_VARIABLE
3 Variable
```

1.7. Regla Modificada: *docstring-min-length*

Exige escribir un docstring en funciones y clases exigiendo un tamaño mínimo. Hemos considerado esta regla interesante ya que obliga al programador a explicar implementaciones de su código.

Regla Original: `docstring-min-length=1`

Regla Modificada: `docstring-min-length=10`

Usos Correctos

```
1 class TransactionManager:
2     """It deals with transactions reading and validating operations
3         """
4     pass
5
6     def validate_iban(self, iban: str):
7         """A method that checks if an iban is correct or not"""
8         pass
```

Usos Incorrectos

```
1 class TransactionManager:
2
3     def validate_iban(self, iban: str):
```

1.8. Regla Modificada: *max-args*

Limita el número máximo de argumentos que puede recibir una función o un método. Creemos que el valor por defecto, 5, es muy limitante y lo hemos aumentado.

Regla Original: `max-args=5`

Regla Modificada: `max-args=6`

Usos Correctos

```
1 def(a, b, c, d, e, f):  
2     pass
```

Usos Incorrectos

```
1 def f(a, b, c, d, e, f, g):  
2     pass
```

1.9. Regla Modificada: *max-line-length*

Limita el número máximo de caracteres que puede haber en una línea. Debido a que trabajamos con monitores más grandes hemos decidido aumentar ese máximo.

Regla Original: max-line-length=100

Regla Modificada: max-line-length=120

Usos Correctos

```
// Este es un comentario de exactamente 120 caracteres. Asegúrate de contar bien para  
que no pase el límite establecido.
```

Usos Incorrectos

```
// Este es un comentario que supera con creces los 120 caracteres porque sigue extendiéndose  
más allá del límite definido para ilustrar el caso correctamente.
```

1.10. Regla Modificada: *single-line-if-stmt*

Permite escribir un if en una sola línea. Esta opción nos resulta interesante para permitir escribir así funciones if que son muy cortas.

Regla Original: single-line-if-stmt=no

Regla Modificada: single-line-if-stmt=yes

Usos Correctos

```
1 | if len(iban) != 24 or iban[:2] != 'ES': return False
```

Usos Incorrectos

```
1 | if x > other.x and y > other.y: for i in range(0, x): z = 2*y
```

1.11. Regla Modificada: *logging-format-style*

Obliga a escribir los logs por pantalla de acuerdo al viejo o nuevo estilo de python. En nuestro caso hemos decidido establecer esta regla para obligarnos a usar el nuevo estilo de python para hacer logs que es más claro.

Regla Original: logging-format-style=old

Regla Modificada: logging-format-style=new

Usos Correctos

```
1 | print(f'Posicion: {x}, {y}')
```

Usos Incorrectos

```
1 | print("Posicion: x = {}, y = {}".format(x, y))
```

1.12. Regla Modificada: *max-nested-blocks*

Restringe el número máximo de bloques anidados permitidos. Hemos decididos bajar el máximo por defecto porque consideramos que permite códigos potencialmente más complejos de leer.

Regla Original: max-nested-blocks=5

Regla Modificada: max-nested-blocks=4

Usos Correctos

```
1 def ejemplo_funcion(a):  
2     if(a > 15):  
3         print(a)
```

Usos Incorrectos

```
1 def ejemplo_funcion(a):  
2     if(a > 3):  
3         if(a > 6):  
4             if(a > 9):  
5                 if(a > 12):  
6                     if(a > 15):  
7                         print(a)
```

1.13. Regla Modificada: *useless-return*

Activa una advertencia cuando hay una declaración de return innecesaria en una función. La hemos incluido para que el código sea lo más compacto y limpio posible, y no incluya elementos innecesarios.

Regla Original: enable=

Regla Modificada: enable=useless-return

Usos Correctos

```
1 def my_function() -> None:  
2     print("Hola")  
3     return
```

Usos Incorrectos

```
1 def my_function() -> None:  
2     print("Hola")
```

1.14. Regla Modificada: *bare-except*

Esta opción activa una advertencia al detectar el uso de bloques except sin especificar una excepción concreta. La hemos considerado puesto que es una buena práctica especificar el error que queremos capturar, para evitar malinterpretaciones a la hora de loggear errores.

Regla Original: enable=

Regla Modificada: enable=bare-except

Usos Correctos

```
1  try:
2      x = 1 / 0
3  except ZeroDivisionError:
4      print("Error: Division por cero")
```

Usos Incorrectos

```
1  try:
2      x = 1 / 0
3  except:
4      print("Se ha producido un error")
```

1.15. Regla Modificada: *max-branches*

Limita el número de bifurcaciones del código (con condicionales del tipo if, elif, else...). Esto lo hemos modificado ya que consideramos que 12 son demasiadas bifurcaciones y no queda un código limpio, por lo que sería mejor utilizar un diccionario y otra estructura de datos más elegante.

Regla Original: max-branches=12

Regla Modificada: max-branches=8

Usos Correctos

```
1 def analizar_datos(valor: int) -> str:
2     valores = {
3         1: "Uno",
4         2: "Dos",
5         3: "Tres",
6         4: "Cuatro",
7         5: "Cinco",
8         6: "Seis",
9         7: "Siete",
10        8: "Ocho",
11    }
12    return valores.get(valor, "Otro")
```

Usos Incorrectos

```
1 def analizar_datos(valor: int) -> str:
2     if valor == 1:
3         return "Uno"
4     elif valor == 2:
5         return "Dos"
6     elif valor == 3:
7         return "Tres"
8     elif valor == 4:
9         return "Cuatro"
10    elif valor == 5:
11        return "Cinco"
12    elif valor == 6:
13        return "Seis"
14    elif valor == 7:
15        return "Siete"
16    elif valor == 8:
17        return "Ocho"
18    else:
19        return "Otro"
```

2. EVALUACIÓN DE LA NORMATIVA Y DEL CÓDIGO

2.1. Capturas de pylint con código sin cumplir normativa

```
PyLint found 1 error, 12 warnings, 57 conventions, 8 refactors in 5 files
TransactionManagementException.py : 4 conventions
    Final newline missing (12:0) [missing-final-newline]
    Missing module docstring (1:0) [missing-module-docstring]
    Module name "TransactionManagementException" doesn't conform to snake_case naming style (1:0) [invalid-name]
    Missing class docstring (1:0) [missing-class-docstring]
TransactionManager.py : 1 warning, 11 conventions, 1 refactor
    Missing module docstring (1:0) [missing-module-docstring]
    Module name "TransactionManager" doesn't conform to snake_case naming style (1:0) [invalid-name]
    Missing class docstring (5:0) [missing-class-docstring]
    Class name "transactionManager" doesn't conform to "[A-Z][a-z]+(?:[A-Z][a-z]+)*$" pattern (5:0) [invalid-name]
    Method name "__init__" doesn't conform to "[a-z][a-z0-9_]*$" pattern (6:4) [invalid-name]
    Missing function or method docstring (26:4) [missing-function-docstring]
    Method name "ReadproductcodefromJSON" doesn't conform to "[a-z][a-z0-9_]*$" pattern (26:4) [invalid-name]
    Using open without explicitly specifying an encoding (29:17) [unspecified-encoding]
    Variable name "DATA" doesn't conform to "[a-z][a-z0-9_]*$" pattern (30:16) [invalid-name]
    Variable name "T_FROM" doesn't conform to "[a-z][a-z0-9_]*$" pattern (38:12) [invalid-name]
    Variable name "T_TO" doesn't conform to "[a-z][a-z0-9_]*$" pattern (39:12) [invalid-name]
    Variable name "TO_NAME" doesn't conform to "[a-z][a-z0-9_]*$" pattern (40:12) [invalid-name]
    Unnecessary "else" after "raise", remove the "else" and de-indent the code inside it (44:8) [no-else-raise]
```

Fig. 2.1. Output tras ejecutar pylint->check project 1/4.

```
TransactionRequest.py : 2 warnings, 18 conventions
    Missing module docstring (1:0) [missing-module-docstring]
    Module name "TransactionRequest" doesn't conform to snake_case naming style (1:0) [invalid-name]
    Missing class docstring (5:0) [missing-class-docstring]
    Attribute name "__IBANFrom" doesn't conform to snake_case naming style (8:8) [invalid-name]
    Attribute name "__IBANTO" doesn't conform to snake_case naming style (9:8) [invalid-name]
    Attribute name "__timeStamp" doesn't conform to snake_case naming style (11:8) [invalid-name]
    Attribute name "recipientName" doesn't conform to snake_case naming style (21:8) [invalid-name]
    Method name "__init__" doesn't conform to "[a-z][a-z0-9_]*$" pattern (6:4) [invalid-name]
    Argument name "IbAnFROM" doesn't conform to "[a-z][a-z0-9_]*$" pattern (6:23) [invalid-name]
    Argument name "IbAnTO" doesn't conform to "[a-z][a-z0-9_]*$" pattern (6:32) [invalid-name]
    Argument name "RECIPIENTName" doesn't conform to "[a-z][a-z0-9_]*$" pattern (6:40) [invalid-name]
    Method name "__str__" doesn't conform to "[a-z][a-z0-9_]*$" pattern (13:4) [invalid-name]
    Attribute name "recipientName" doesn't conform to snake_case naming style (17:4) [invalid-name]
    Attribute name "recipientName" doesn't conform to snake_case naming style (20:4) [invalid-name]
    Attribute name "IBAN_FROM" doesn't conform to snake_case naming style (24:4) [invalid-name]
    Attribute name "IBAN_FROM" doesn't conform to snake_case naming style (27:4) [invalid-name]
    Attribute name "IBAN_TO" doesn't conform to snake_case naming style (31:4) [invalid-name]
    Attribute name "IBAN_TO" doesn't conform to snake_case naming style (34:4) [invalid-name]
    Unused private member 'TransactionRequest.__recipient' (7:8) [unused-private-member]
    Unused private member 'TransactionRequest.__timeStamp' (11:8) [unused-private-member]
__init__.py : 3 conventions
    Final newline missing (3:0) [missing-final-newline]
    Missing module docstring (1:0) [missing-module-docstring]
    Module name "UC3MMoney" doesn't conform to snake_case naming style (1:0) [invalid-name]
```

Fig. 2.2. Output tras ejecutar pylint->check project 2/4.

```
main.py : 1 error, 9 warnings, 21 conventions, 7 refactors
Line too long (158/120) (9:0) [line-too-long]
Unnecessary parens after 'if' keyword (21:0) [superfluous-parens]
Unnecessary parens after 'if' keyword (22:0) [superfluous-parens]
Unnecessary parens after 'if' keyword (23:0) [superfluous-parens]
Unnecessary parens after 'if' keyword (24:0) [superfluous-parens]
Unnecessary parens after 'if' keyword (25:0) [superfluous-parens]
Final newline missing (88:0) [missing-final-newline]
Missing module docstring (1:0) [missing-module-docstring]
No name 'transactionManager' in module 'UC3MMoney' (1:0) [no-name-in-module]
Constant name "letters" doesn't conform to UPPER_CASE naming style (5:0) [invalid-name]
Constant name "shift" doesn't conform to UPPER_CASE naming style (6:0) [invalid-name]
Too many arguments (7/6) (11:0) [too-many-arguments]
Too many positional arguments (7/5) (11:0) [too-many-positional-arguments]
Redefining name 'f' from outer scope (line 11) (11:21) [redefined-outer-name]
Unused argument 'a' (11:6) [unused-argument]
Unused argument 'b' (11:9) [unused-argument]
Unused argument 'c' (11:12) [unused-argument]
Unused argument 'd' (11:15) [unused-argument]
Unused argument 'e' (11:18) [unused-argument]
Unused argument 'f' (11:21) [unused-argument]
Unused argument 'g' (11:24) [unused-argument]
Formatting a regular string which could be an f-string (17:10) [consider-using-f-string]
Useless return at end of function or method (14:0) [useless-return]
Missing function or method docstring (20:0) [missing-function-docstring]
Too many nested blocks (5/4) (21:4) [too-many-nested-blocks]
Unused variable 'x' (29:8) [unused-variable]
Missing function or method docstring (33:0) [missing-function-docstring]
```

Fig. 2.3. Output tras ejecutar pylint->check project 3/4.

```
Unnecessary "elif" after "return", remove the leading "el" from "elif" (34:4) [no-else-return]
Too many return statements (9/6) (33:0) [too-many-return-statements]
Too many branches (9/8) (33:0) [too-many-branches]
Function name "Encode" doesn't conform to '[a-z][a-z0-9_]*$' pattern (53:0) [invalid-name]
Function name "Decode" doesn't conform to '[a-z][a-z0-9_]*$' pattern (63:0) [invalid-name]
Missing function or method docstring (74:0) [missing-function-docstring]
Variable name "strRes" doesn't conform to '[a-z][a-z0-9_]*$' pattern (78:8) [invalid-name]
Unnecessarily calls dunder method __str__. Use str built-in function. (78:17) [unnecessary-dunder-call]
Variable name "EncodeRes" doesn't conform to '[a-z][a-z0-9_]*$' pattern (80:8) [invalid-name]
Variable name "DecodeRes" doesn't conform to '[a-z][a-z0-9_]*$' pattern (82:8) [invalid-name]
standard import "string" should be placed before first party import "UC3MMoney.transactionManager" (2:0) [wrong-import-order]
```

Fig. 2.4. Output tras ejecutar pylint->check project 4/4.

2.2. Captura de pylint con código cumpliendo normativa

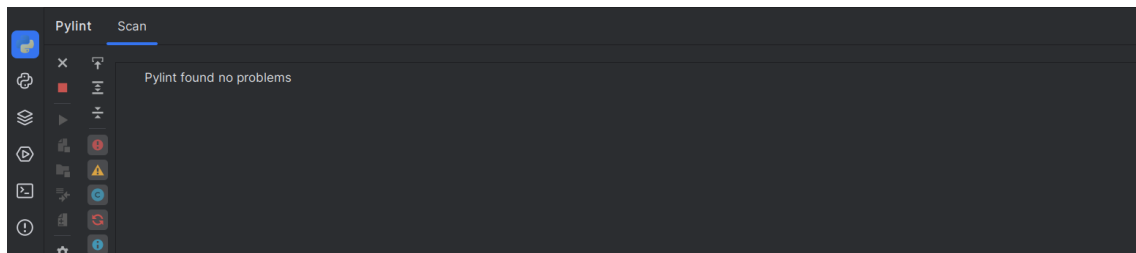


Fig. 2.5. Output tras ejecutar pylint->check project tras refactorizar el código y respetar la normativa establecida.

2.3. Evaluación del código

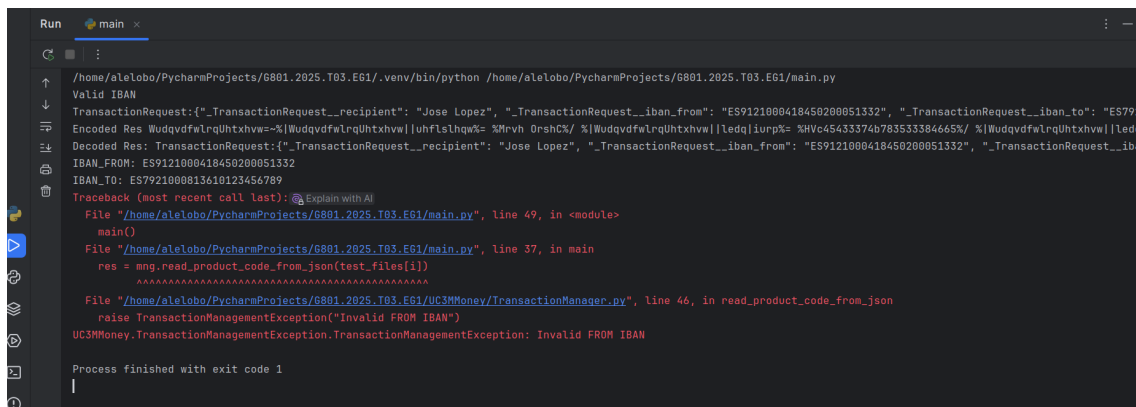


Fig. 2.6. El siguiente output del main.py muestra en primer lugar el resultado en pantalla tras ejecutar una transacción con un IBAN válido y seguidamente muestra por pantalla el error que se notifica en caso de que se realice una transacción con un IBAN incorrecto.